Recommender Systems - Course Project Report

Cumulative report template - Parts 1, 2, and 3

Team	14
Members	Mathis Fournier, David Ramos, Paul Lorenz
For Part 3 track: select one	A / B / C
GitHub repository link	https://github.com/DavidRamosSal/recommender-systems-project

How to use this template

- *This is one cumulative report. Extend and revise the same document at each project stage.*
- *Maximum length: 4 pages at Part 1, 8 pages at Part 2, and 12 pages for the final report, excluding references.*
- *Keep the section headings below. You may add subsections where useful.*
- *All explanations and justifications requested in the project specification belong in this report.*
- *The Results Sheet contains the complete standardized numerical record. In the report, include only the tables and figures needed to support your analysis.*
- *Delete all italic instructional text before the final submission.*

1. Part 1 - Data, Protocol and Baselines

1.1 Data characterization

There are 6040 users as described in the project docs. All entries are non-null and all user_ids are unique. There are 3883 unique movies, of which 3706 have at least one rating and 177 have none. The 1,000,209 ratings span 2000-04-25 to 2003-02-28 (UTC).

The density is the ratio of the number of ratings available to the total number of possible ratings there can be. It is 4.47% for the 6,040 users and 3,706 rated movies, or 4.26% including all 3,883 movies.

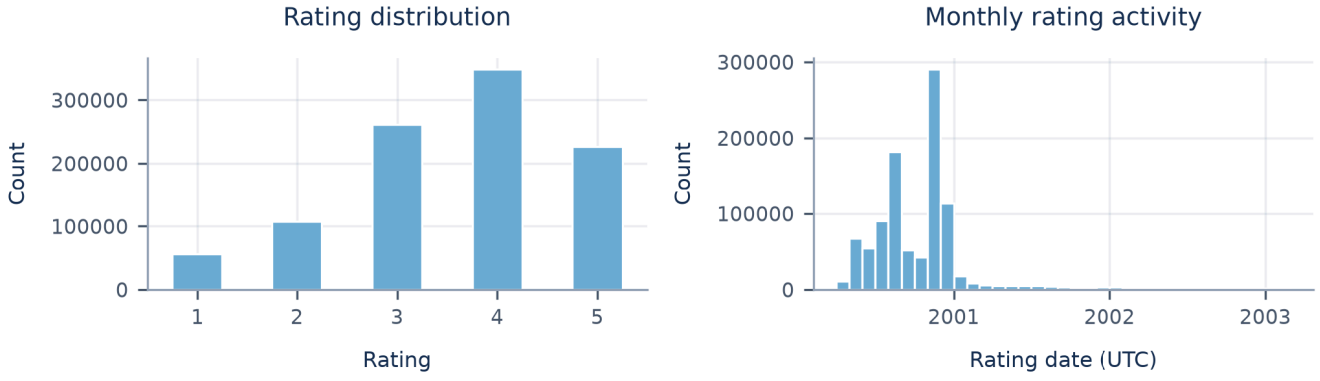


Figure 1. Rating distribution and monthly rating activity (UTC).

In terms of rating distribution (Figure 1), low ratings seem under represented. In the temporal distribution of the rating counts we see most of the ratings happened in the year 2000, the burstiness referenced in the project description. There is a great drop for ratings in 2001 and the rating counts continue to decrease slowly with time.

We see a very skewed distribution of ratings per user (Figure 2, top row; Table 1): most users rank a small portion of the total movies, with more than 75% of users rating less than 250 out of the 3883 movies, while some rank more than 1000. For items (Figure 2, bottom row), we see again a very skewed distribution: with 75% of the items having 330 ratings or fewer, while a few items are ranked over 2000 times.

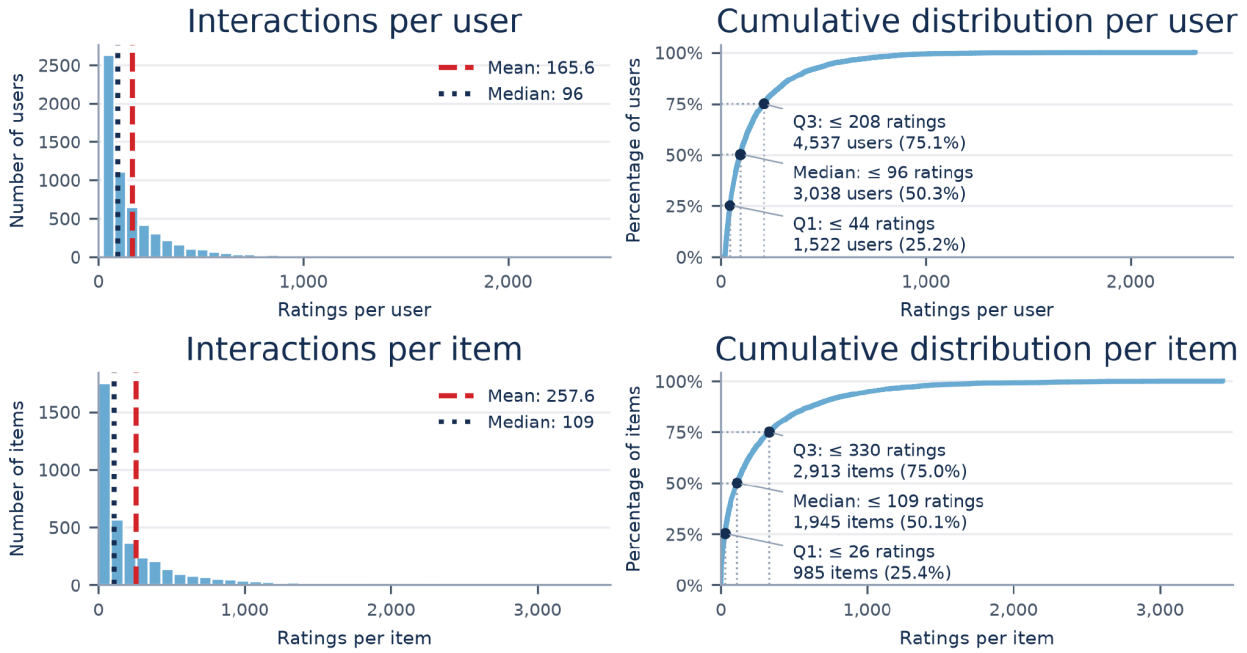


Figure 2. Interactions per user (top) and item (bottom): histograms (left), cumulative distributions (right); Lecture 5.1, slide 5.

Table 1. Interaction counts per user and per item.

Interactions	Minimum	Median	Q3 (75%)	Maximum	Mean
Per user	20	96	208	2,314	165.6
Per item	0	109	330	3,428	257.6

1.2 Preprocessing and feedback treatment

Filtering: We do not filter any users or movies, so 0 ratings are removed. Every user already has at least 20 ratings. A minimum of 5 training ratings per movie would remove only 0.07 % of the ratings, but 9 % of the catalogue, and it would remove the long-tail movies we want to keep.

Feedback: A movie is liked if its rating is 4 or 5 (about 58 % of ratings). We tested two user profiles on validation: liked movies only, or all rated movies. Liked movies only worked better for every similarity, because a rating of 1 to 3 does not show that the user likes the movie.

1.3 Train/validation/test split

We use a global temporal split with the same cut dates for all users. The model never learns from ratings that come after the ones it is tested on. We compared three options. Table 2 shows what each one gives.

Table 2. Comparison of global temporal split options.

Option	Cut dates (validation / test start)	Ratings (train / val / test)	Users we can test (val / test)	Ratings from users with no history (val / test)
A: 80/10/10 by number of ratings	2000-12-02 / 2000-12-29	800k / 100k / 100k	492 / 1,159	76 % / 4 %
B: val = Dec 2000, test = 2001 onwards	2000-12-01 / 2001-01-01	791k / 113k / 95k	538 / 1,150	75 % / 1 %

C (kept): val = Jan to Jun 2001, test = Jul 2001 onwards	2001-01-01 / 2001-07-01	905k / 47k / 48k	892 / 777	0.4 % / 1 %

What we kept, option C: Train is 2000-04-25 to 2000-12-31 (904,757 ratings, 90.5 %). Validation is 2001-01-01 to 2001-06-30 (47,314 ratings, 4.7 %). Test is 2001-07-01 to 2003-02-28 (48,138 ratings, 4.8 %).

Why not A or B: Most ratings (90.5 %) were made in 2000, and 63 % of users enter all their ratings within 24 hours. A time cut therefore puts most users on one side, and only users who come back later can be tested. A and B cut inside or right after this burst. In both, about three quarters of the validation ratings come from new users with no history, so only 492 and 538 users can be tested, and they are not like the users in test. With C, validation and test are both after the burst. They have the same size and the same kind of users, so the settings chosen on validation should also work on test. We chose the dates before running any model.

However, A and B give more test users (about 1,150 against 777). We think a validation set that looks like the test set is worth more. We only test on returning users, who are more active than average (median history of 169 to 212 ratings, against 96 for all users). For the final test, we train again on train + validation.

1.4 Evaluation protocol

- **Candidate set:** All 3,706 rated movies, except the ones the user already rated. We rank the full catalogue, as the project asks.
- **Liked movies and tested users:** A liked movie is rated 4 or 5. We test users with at least one earlier rating and one liked movie in the period: 892 in validation and 777 in test. Metrics are averaged over these users.
- **Metrics:** Precision@10, Recall@10, NDCG@10 and catalogue coverage.

1.5 Baseline methods and settings

1.5.1 Item-based kNN

Each movie keeps its k most similar movies. The score of a movie for a user is the sum of its similarities to the movies the user liked, among these k neighbours. We remove movies already rated and recommend the top 10. We tested 60 settings on validation: 3 similarities, 2 user profiles and 10 values of k (from 5 to 2000). Table 3 shows the best k for each similarity.

Table 3. Best item-kNN settings and validation NDCG@10.

Similarity	Liked movies: best k	NDCG@10	All ratings: best k	NDCG@10
Cosine	500	0.1735	1000	0.1397
Jaccard	200	0.1659	300	0.1294
Adjusted cosine	50	0.1641	50	0.1514

Selected: cosine, liked movies only, k = 500, because it has the best NDCG@10 on validation. Other values of k between 200 and 1000 give almost the same score. With a large k, the model probably recommends the same popular movies to many users: coverage falls from 20 % (k = 5) to 7 % (k = 2000).

1.5.2 Most Popular

Every movie is scored by how often it occurs in the training data, and every user receives the same ranking, minus the movies they have already rated. The project leaves two choices open, and we evaluated both: the time window (all-time popularity vs. only the last 30, 90 or 180 days before the end of the training data) and the feedback (all ratings vs. only ratings ≥ 4, the same definition of “liked” as in

Section 1.2). This gives eight variants. Ties are broken by all-time count. We select the variant with the best validation NDCG@10, which is the last 30 days with ratings ≥ 4, and also report all-time popularity with ratings ≥ 4 as the second variant. As for item-kNN, the final model for the test is refit on train + validation, so the window ends right before the test period. Most Popular is deterministic, so no model seed is needed. The 95 % confidence intervals come from a bootstrap over users (1,000 samples, seed 42; Lecture 3.1, slide 10).

1.6 Results and analysis

On the test set, item-kNN is the better baseline (Figure 3): it reaches an NDCG@10 of 0.2075, compared with 0.162 for the selected Most-Popular variant (last 30 days, ratings ≥ 4) and 0.177 for all-time popularity. Both models find about the same share of relevant movies (recall 0.072), but kNN's top 10 contains more hits (precision 0.183 vs. 0.125). But the gap is smaller than it may seem at first sight. On validation, kNN scores 0.174 and Most Popular 0.161, and kNN falls inside Most Popular's confidence interval (0.148–0.175). Simply recommending popular movies is hard to beat on this dataset. The large best k for kNN (500) points the same way, because bigger neighbourhoods push kNN towards popular movies. The biggest difference is coverage: kNN recommends almost 10 % of the catalogue, Most Popular only 2–4 %, because everyone gets the same few hits. The 30-day window was best on validation but not on test. In December 2000 that window holds 107,762 ratings, in June 2001 only 4,981. Popularity based on so few ratings is unreliable, this is a side effect of the burst. We still keep the variant chosen on validation instead of switching based on test results.

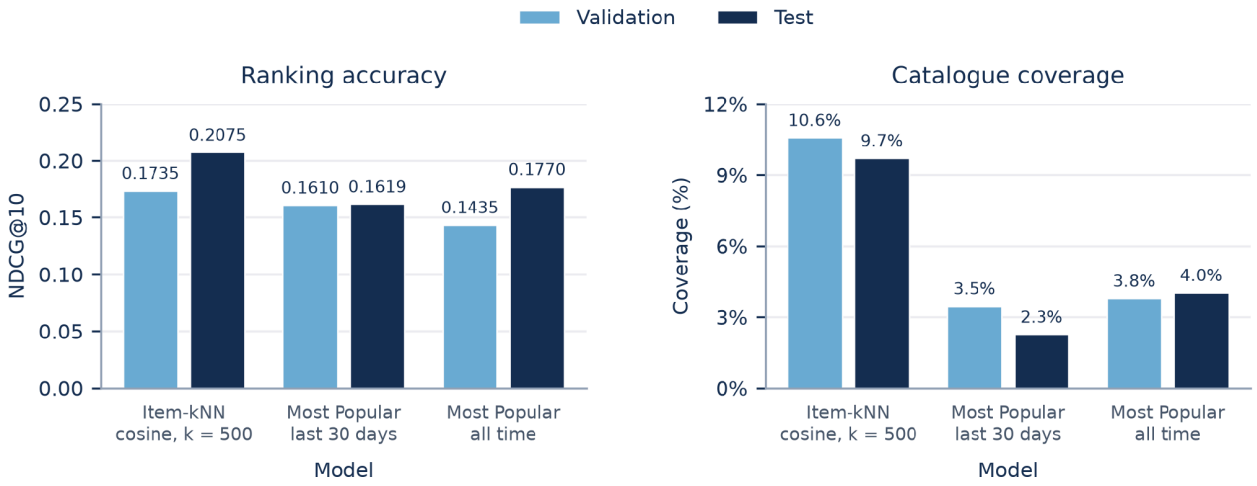


Figure 3. Validation and test NDCG@10 and catalogue coverage. All models use ratings ≥ 4; test models are refit on train + validation.

2. Part 2 - Models and Honest Comparison

2.1 Models

Describe the additional models and paradigms. Clearly identify the self-implemented algorithm and explain its implementation.

2.2 Hyperparameter tuning and variability

Describe the validation-only tuning procedure, search spaces/budgets, random seeds, and confidence intervals or statistical testing.

2.3 Overall results

Present and discuss the main comparison. Explain important differences and avoid unsupported claims.

2.4 Disaggregated analysis

Report and interpret results broken down by at least two of the required dimensions: items with few vs. many training interactions, low- vs. high-activity users, popular vs. long-tail items, and early vs. late test period. Define any thresholds used and focus on findings that change or qualify the overall conclusions.

2.5 Sparsity ablation

Report and discuss the 100%, 50%, and 20% training-interaction experiments, including changes in the ranking between models.

3. Part 3 - Beyond Accuracy

3.1 Selected track and motivation

State Track A, B, or C and briefly motivate the selected intervention/extension.

3.2 Method

Describe only your selected track. The Track A/B/C guidance below is provided as instructions; delete the guidance for the two tracks you did not select before submission.

If Track A - Diversity and popularity bias

State the base recommender, selected re-ranking method, genre-vector item representation, trade-off parameter, and exact item-distance measure. Describe how the parameter was swept.

If Track B - Fairness

Define the release-decade provider proxy, report and justify the final decade boundaries and group sizes, state and justify the adopted fairness notion, describe the consumer groups based on MovieLens user attributes, and identify the stage and form of the mitigation.

If Track C - Group recommendations

State the selected synthetic group type and justify the chosen group size (3-5). Describe the three aggregation strategies. Identify which strategy was extended to the sequential scenario and explain how satisfaction is carried forward between recommendation rounds.

3.3 Results and trade-offs

Present the required track-specific results. Track A: report intra-list diversity, catalogue coverage, Gini and calibration error across the parameter sweep, and discuss the accuracy-diversity frontier. Track B: report position-weighted exposure against catalogue share, per-group consumer-side accuracy, and the fairness-performance frontier. Track C: compare group satisfaction, disagreement and minimum member satisfaction across the three aggregation strategies, then analyse how member and group satisfaction develop in the sequential scenario.

3.4 Cost and interpretation

State what is gained, what is lost, and who bears the cost where applicable. Explain what operating choice or strategy you would select and why.

3.5 "One number we cannot explain"

Identify one result you do not fully understand and describe how you would investigate it.

4. Discussion and Conclusions

Summarize the main findings across the project. Discuss limitations, failed approaches where relevant, and what the experiments support.

5. Contribution Statement

For each team member, provide two or three sentences describing their contribution.

6. Use of AI Assistants

State which AI tools were used and for what. If none were used, state that explicitly.

References

List references used in the report. References do not count toward the page limit.